Tidekeeper

Niagara at Scale / Track B / One-page proposal / October 26, 2026

Mechanic. A top-down, one-level rescue game. Move a beacon, attract a school, or drive it with a repel pulse. Rescue 70% of the agents into a sanctuary within 120 seconds. Agent positions determine rescue; progress changes the score and win/lose state.

Two configurations. GPU: 100,000+ sprite particles using separation, alignment and cohesion with a spatial neighbor grid. CPU: 500-5,000 Niagara mesh agents with grazing, fleeing and following states, a perception cone, leader roles and obstacle avoidance. Switch configurations and change counts at runtime.

Budget. Declare 1920x1080 at 60 fps / 16.6 ms and Niagara GT+RT at most 4 ms. Use p95 frame time to assess sustained frame pacing. Start at 500 CPU agents and 100,000 GPU agents, then sweep counts upward to identify failure. Development-laptop data must be distinguished from lab-machine validation.

Data reaches gameplay. C++ supplies beacon position, attraction and pulse controls to a custom Niagara data interface. CPU occupancy returns directly. GPU rescue and overflow totals are reduced into an 8-byte summary and transferred asynchronously through Niagara's GPU readback manager. Run IDs protect restarts from stale results.

Measurement and optimization. Capture at least five counts per configuration with stat Niagara, stat GPU, Niagara Debugger and Unreal Insights. Provide a console count control. Compare CPU all-pairs and spatial-hash searches before/after under the same seeded test route; record frame, CPU and GPU times and explain density-dependent limits.

References. Borrow indirect flock guidance from Sheepherds! and large, responsive fish schools from ABZU. Official descriptions and trailer links are in ReferenceNotes.md. Personal reference viewing/playing must be completed by the student; no viewing session is asserted here.

Three-day scope. Day 1: playable rescue loop, both Niagara configurations and reliable data return. Day 2: count sweeps, measured optimization and runtime checks. Day 3: packaged Windows build, tagged Git release, a maximum two-page performance report and a 60-90-second captured demo.